

PungentFunk Utilities

Architecture and Design Principles Bible

Latest audit edition - regenerated from the most recent uploaded PungentFunkUtilities.zip

Current archive audit

Metric	Current result
C# scripts	94
Editor scripts	56
Runtime/shared scripts	38
Total C# lines	30,404
Assembly definition files	0
Files with namespace declarations	0
MenuItem declarations	77
CreateAssetMenu declarations	12
InitializeOnLoad declarations	3
EditorWindow classes	27

Primary purpose: establish durable architecture rules, UI standards, package boundaries, interoperability models, access patterns, layout principles, audit snapshot, and a current script index that should guide future PungentFunk Utilities development.

Latest delta note: compared with the previous indexed design bible, this uploaded archive exposes the same 94 C# script paths. The current audit updates line counts, menu counts, hotspot counts, and the generated index from the uploaded archive contents.

1. Scope and North Star

PungentFunk Utilities should be treated as a family of Unity editor/runtime utility assets, not a single monolithic toolbox. Each lab or suite should be independently releasable, while the larger bundle should expose a cohesive control panel, shared visual language, and optional cross-lab enhancements.

The core promise is simple: automate tedious game-development workflows without adding new tedium. Every tool should be understandable at a glance, safe to try, and powerful enough to scale through presets, profiles, contextual help, and visual feedback.

Release model

- Core package: shared registry, lab launcher, theme system, preferences, common editor UX helpers, optional integration contracts, and documentation patterns.
- Standalone lab packages: independently useful products such as Colour Lab, Audio Lab, Asset Placement Lab, Texture Lab, Scene Workflow Lab, Asset Preview and Export Lab, Project Audit and Authoring Lab, Content Generation Lab, and UI and Feedback Lab.
- Bundle package: installs all labs and unlocks cross-lab linking, shared dashboards, richer context menus, and handoff workflows between tools.
- Project adapters: thin game-specific wrappers that pre-fill generic utility settings, preserve old menu paths, or translate project-specific data into generic profiles without polluting the generic package.

Definition of a PungentFunk utility

- It solves a broadly reusable Unity workflow problem.
- It uses generic names, generic data models, and user-configurable presets instead of hardcoded project assumptions.
- It works in a clean project, or clearly declares optional dependencies that can be absent without compile/runtime errors.
- It has preview/dry-run behavior before destructive changes and records Undo where scene or asset changes are made.
- It uses the shared visual language and persistence helpers unless there is a deliberate, documented reason not to.

2. Product Architecture

Layer	Purpose	Rules
PungentFunk Utilities Core	Registry, launcher/control panel, lab menu routing, shared theme, preferences, shared editor widgets, package status metadata.	Every lab may depend on Core. Core must not depend on any lab. Keep Core small, stable, and boring.
Runtime model assemblies	ScriptableObjects, MonoBehaviours, data models, adapters, and runtime services used outside the Unity Editor.	No UnityEditor references. No editor-only menu logic. Use stable IDs and version fields for serialized data.
Editor lab assemblies	Editor windows, inspectors, scanners, generators, validators, and authoring panels.	May depend on UnityEditor and Core. Scan and apply operations must be explicit, cancelable where expensive, and cached.
Optional bridge/adapters	Code that connects two labs or a lab to third-party packages.	Must be removable. Use version defines, reflection, asmdef constraints, or isolated bridge packages to avoid hard dependencies.
Project adapters	Game-specific wrappers and presets outside the generic asset.	Thin only. They may reference game classes, but generic packages must never compile against them.

Dependency direction

- Allowed: Lab -> Core, Editor -> Runtime, Project Adapter -> Generic Lab, Bridge -> Lab A + Lab B.
- Discouraged: Lab A directly compiles against Lab B unless Lab B is declared as a required dependency for that product.
- Forbidden: Core -> Lab, Runtime -> Editor, Generic Lab -> game-specific project class, and cyclic lab dependencies.
- Preferred optional link: detect availability by registry ID, type lookup, define symbol, optional bridge package, or service lookup; show a helpful disabled state when absent.

Labs should own their domain models and workflows. Shared abstractions should move to Core only when at least two labs truly need them and they are stable enough to support as public API. Avoid pre-emptively centralising immature helpers.

3. Editor UI, UX, Layout, and Accessibility Principles

3.1 Window and panel layout design principles

PungentFunk Utilities should share a cohesive visual language, but each tool should not be forced into the same layout pattern. The correct layout must be chosen based on the workflow, information density, user intent, frequency of use, and risk level. A utility layout is part of the product design, not cosmetic decoration.

The suite should draw from Unity editor tooling, professional software, web applications, creative software, infographics, graphic design, poster/editorial design, copywriting, data visualization, and accessibility design. The visual theme should provide cohesion; the window and panel structure should be purpose-built for the task.

3.2 Cohesive identity, flexible layouts

- Consistency should come from shared theme colours, typography rules, spacing scale, icons, status language, button styles, safety patterns, registry metadata, and lab navigation.
- Consistency should not mean every utility uses the same panel arrangement, every workflow is forced into a vertical inspector, or every advanced system is hidden behind the same foldout structure.
- The best layout is the one that makes the tool intended workflow obvious, efficient, and pleasant.
- Major tools should feel like focused workspaces. Minor tools should stay compact and direct.
- Do not preserve a layout just because another PungentFunk utility uses it. Copy interaction conventions, not unsuitable information architecture.

3.3 Layouts should be purpose-built

Before implementing a window, identify the primary workflow. The layout should directly support that workflow rather than defaulting to a long inspector-style stack.

- Configuration: the user is tuning settings or presets.
- Scanning: the user is searching scenes, assets, references, coverage, or validation state.
- Comparison: the user is reviewing many candidates or relationships.
- Selection and detail: the user selects an item and edits its properties.
- Preview-first generation: the user iterates on generated visuals, placements, palettes, textures, names, or exports.
- Batch operation: the user needs preview, selection, apply, summary, and possibly undo/repair.
- Monitoring: the user watches runtime/debug state and needs quick filters or toggles.
- Authoring: the user creates data assets, recipes, profiles, rules, matrices, or libraries.

3.4 Supported layout templates

Template	Best for	Design notes
Simple vertical stack	Small configuration tools, one-shot helpers, lightweight settings.	Use header -> settings -> action -> status/help. Avoid for tools needing comparison, preview, or large result sets.
Two-column layout	Source/result, configuration/preview, list/detail, asset selection/output.	Use draggable splitters when both sides compete for space. Persist splitter sizes through <code>UtilityWindowPrefs</code> .
Three-panel layout	Advanced authoring, asset browsers, debug tools, lab hubs.	Typical structure: left navigation/source, centre workspace/results, right details/actions. Collapse or stack panels at narrow widths.
Grid/card layout	Asset sets, palettes, prefab previews, icons, generated names, tool launchers.	Cards should wrap vertically, use compact metadata, and rely on icons, badges, colour chips, and status pills to reduce text clutter.
Table/matrix layout	Coverage tools, validation results, interaction matrices, dependency maps, compatibility checks, token/reference audits.	Use sorting, filtering, grouping, frozen or repeated key labels where possible. Horizontal scrolling is acceptable only when preserving matrix relationships matters.
Canvas/lab workspace	Complex generation tools, asset placement, spatial workflows, procedural design, multi-step creative tools.	Use a central workspace/preview with side configuration, collapsible panels, contextual toolbar, visual overlays, mode-specific controls, and optional docked subpanels.
Wizard/step flow	Setup tools, first-time configuration, migration, export/package preparation, risky multi-stage operations.	Show current step, required/optional inputs, validation status, dry-run results, and final apply/export action.

Template	Best for	Design notes
Dashboard/overview	Control panel, lab home screens, project health summaries, package status, scan summaries, debug/coverage overview.	Surface summaries, warnings, search, pinned/recent tools, lab/module groupings, and quick actions. Do not make dashboards dense control panels.
Contextual mini panel / inline toolbar	Component-specific tools, scene selection tools, terrain helpers, asset quick actions.	Use inside inspectors, overlays, context menus, related PungentFunk windows, or toolbar areas when a full window would add friction.

3.5 Layout selection principles

- Task frequency: common actions should be closer and more visible than rare actions.
- Risk level: destructive or project-wide actions need preview, confirmation, and visual separation.
- Information density: dense information should be grouped, filtered, summarised, and progressively disclosed.
- Comparison needs: if the user must compare items, avoid hiding them behind separate foldouts.
- Preview needs: if visual output matters, give preview space priority.
- Selection flow: if the user selects an item and edits details, use list/detail or grid/detail patterns.
- Window size: layouts must degrade gracefully at smaller widths.
- Scanning speed: use alignment, spacing, icons, and colour to help the user understand state quickly.
- Cognitive load: prefer fewer visible decisions at once, with advanced options collapsed or contextual.
- Creative flow: generation/design tools should support iteration without burying regenerate, lock, save, preview, or apply actions.

3.6 Responsive layout rules

- The window must remain usable at small sizes. Do not assume a wide monitor.
- Prefer vertical scrollbars over horizontal scrollbars. Avoid horizontal scrolling unless the data is inherently matrix-like, timeline-like, graph-like, or canvas-like.
- Use wrapping rows, compact cards, collapsible sections, tabs, and column stacking before resorting to horizontal scrolling.
- Important shared borders between subpanels should be draggable where panel proportions matter; persist dimensions through `UtilityWindowPrefs`.
- Avoid dead space. If a row or card has large blank interior gaps, shrink padding, wrap controls, collapse secondary metadata, or change layout template.

3.7 Progressive disclosure and assistance

- Show the core workflow first.
- Keep advanced options collapsed by default.
- Explain advanced options contextually.
- Avoid hiding essential actions in advanced sections.
- Use short inline descriptions where they prevent confusion.
- Use detailed help foldouts for complex systems.
- Use contextual tooltips for fields, enum options, buttons, icons, and status indicators.

Every meaningful field should have a tooltip or help affordance. Tooltips should explain what the field controls, when to change it, how it differs from related options, what the selected enum option does, whether the value is destructive/experimental/expensive/preview-only, and expected units/ranges/side effects.

Enum fields should provide contextual explanation for the currently selected option. For complex enums, use a small description box below the field or a hover tooltip beside the selector.

3.8 Visual assistance and accessibility

- Use colour chips for palette, audio, material, or surface status.
- Use icons for scan, warning, error, success, lock, refresh, apply, preview, dependency, and missing package states.
- Use graphs for curves, distributions, coverage, relationships, and procedural output where they clarify faster than text.

- Use diagrams for spatial placement, pipeline flows, or complex rules.
- Use mini previews for prefabs, textures, palettes, terrain layers, generated results, and export outputs.
- Use status pills for stable, experimental, deprecated, project-adaptor, missing dependency, and selected states.
- Use legends where colour or symbols carry meaning.
- Do not rely on colour alone. Pair colour with icons, labels, patterns, or tooltips.
- Support readable contrast, keyboard access where practical, scalable text where possible, and visible focus/selection states.

3.9 Layout anti-patterns

- Long undifferentiated walls of fields.
- Excessive nested foldouts.
- Large empty gaps caused by rigid sizing.
- Horizontal scrolling except for justified tables, matrices, timelines, graphs, or canvases.
- Controls that jump around during refresh.
- Important actions hidden below large scroll regions.
- Multiple duplicate buttons doing the same thing.
- Dense text where an icon, swatch, badge, or graph would be clearer.
- Tool windows that assume a large monitor.
- Mixing unrelated workflows in the same panel without grouping.
- Making every utility look identical at the cost of usability.

4. Editor Access and Integration Principles

Menu items are the baseline access method, not the only access method. PungentFunk Utilities should be accessible through the Unity Editor in ways that match the user current context. A tool should appear where the user naturally needs it, while avoiding duplicate clutter.

4.1 Organised menu paths

All utilities should have clean, predictable menu placement. Preferred root paths include Tools/PungentFunk Utilities/..., Tools/PungentFunk Utilities/Labs/..., Core, Scene, Assets, Generation, Colour, Audio, Debug, Project Audit, and Settings. Existing Tools/Utilities paths may remain as current compatibility while final release naming is standardised.

- Avoid duplicate menu entries.
- Avoid multiple names for the same tool unless preserving a useful legacy alias.
- Avoid deep nesting without clear reason.
- Avoid old project-specific menu names in generic utilities.
- Avoid foldout-like menu categories that contain only one item.
- Legacy aliases should open the new generic implementation, not duplicate functionality.

4.2 Utility Control Panel and Lab Hubs

Every significant utility should register with the central PungentFunk registry and be discoverable through the main control panel. Larger suites should prefer lab hub entry points over scattered menu items.

- Search, lab filtering, module filtering, status filtering, recently used tools, pinned/favourite tools, stable/experimental/deprecated indicators, missing optional dependency indicators, lab descriptions, direct open actions, and context-aware launch actions where possible.

4.3 Additional editor access methods

- Unity Editor toolbar access: frequent tools may expose compact quick-launch buttons, dropdowns, recent tools, mode toggles, debug/preview toggles, or status indicators. Do not duplicate the whole menu tree.
- Scene View Overlay menu integration: scene-related utilities should use proper Scene View overlay menu entries or overlay-integrated controls, not unrelated floating windows pretending to be overlays.

- Right-click context menus: expose concise actions on relevant scene objects, components, assets, folders, materials, textures, prefabs, ScriptableObjects, terrains, audio assets, palette assets, placement groups, and generated markers.
- Inspector-embedded toolbars: related components/assets may expose small toolbar rows, validation/status panels, quick actions, Open in Lab buttons, Scan/Validate/Repair buttons, mini previews, and dependency messages.
- Integration inside other PungentFunk utilities: expose relevant cross-lab features through optional services, not hard dependencies.
- Shortcut and command palette access: frequently used actions should support keyboard shortcuts or future command-palette discovery where appropriate.
- Selection-aware launching: tools should detect current Unity context when opened and preload selected terrain, palette, prefabs, scene objects, audio profiles, placement groups, or related assets where relevant.
- Quick actions and recent tools: support recently opened utilities, pinned tools, favourite labs, last active module per lab, resume last session, open related tools, and continue where I left off affordances through shared preferences.

5. Registry, Navigation, and Lab Hubs

The registry is the package-level discovery contract. Every utility should have an ID, display name, lab, module, category, sort order, status, description, tags, menu path, capability flags, and optional related utility IDs. Lab hubs should organise tools without removing direct window access.

Field	Purpose
Id	Stable machine-readable identifier. Use kebab-case. Never reuse for a different tool.
DisplayName	User-facing name. Generic, clear, and asset-store friendly.
Lab / Module / Category	Primary navigation grouping. Lab is product area; module is subdomain; category is menu/search taxonomy.
Description	One-sentence promise: what the tool does and when to use it.
PackageStatus	Stable, Experimental, In Progress, Deprecated, or Project Adapter. Displayed in launcher and docs.
RelatedUtilityIds	Optional links to complementary tools. Must not imply a hard dependency unless explicitly declared.
Capability flags	Supports selection, context menu, scene overlay, batch actions, lab hub behaviour, toolbar shortcut, or inspector integration.

6. Optional Dependency and Cross-Utility Integration Principles

PungentFunk Utilities should be modular enough for each lab or suite to ship independently, while also becoming more powerful when multiple labs are installed together. Each utility should be designed with required dependencies, optional integrations, and suggested companion utilities.

6.1 Dependency levels

- Required dependencies: dependencies that must exist for the tool to compile and function. Keep these minimal and documented clearly.
- Optional integrations: features that activate when another PungentFunk lab or compatible package is installed. They must not produce compile errors when the related lab is absent.
- Suggested companion utilities: tools that enhance a workflow but are not required. Missing companion messages should be helpful and non-blocking.

6.2 Optional integration implementation patterns

- Assembly definition optional references.
- Version defines.
- Reflection-safe discovery.
- Interface packages.
- Adapter classes in separate integration folders.
- Registry-based service lookup.
- ScriptableObject-based loose references.

- Conditional compilation symbols.
- Soft dependency metadata in utility descriptors.

6.3 Dependency tracking

Maintain a running dependency/interoperability map for each utility or lab. Track required PungentFunk systems, optional integrations, external Unity package dependencies, version requirements, runtime/editor-only status, standalone-shipping status, full-suite-only status, exposed services, and consumed services. This map should inform package splitting, asmdef boundaries, menus, documentation, QA, and future architecture.

6.4 Current and recommended linkage ledger

Lab / utility	Required base	Optional hooks / enhancement points
Utility Core	None	Discovers all installed labs; may show status and missing optional bridge packages.
Debug Lab	Core	Scene Workflow for gizmo sources and navigation; Project Audit for debug coverage reports; UI Feedback for runtime notifications.
Asset Placement Lab	Core; Placement runtime models	Colour Lab for palette-driven prefab/material variation; Texture Lab for density masks; Scene Workflow for navigation and handles; Project Audit for validation/repair; Audio Lab for surface metadata.
Scene Workflow Lab	Core	Debug Lab for gizmo diagnostics; Asset Placement for path/placement authoring handoff.
Colour Lab	Core; Colour runtime models	Texture Lab for palette-to-texture masks; Asset Placement for material variation; UI Feedback for theme previews.
Texture Lab	Core; Texture generation models	Colour Lab for palette sources; Asset Preview/Export for previews and asset packaging.
Audio Lab	Core; Audio runtime models	Project Audit for coverage validation; Asset Placement/Scene Workflow for contextual surface tagging; future Settings Lab for mixer/runtime config.
Asset Preview and Export Lab	Core	Colour Lab for colourway icons; Appearance/UI/project adapters for product-specific icon generation.
Project Audit and Authoring Lab	Core	Can inspect other installed labs through optional scan modules; should not own their runtime models.
Content Generation Lab	Core	Project Authoring Lab for batch asset generation; UI Feedback for token preview/presentation.
UI and Feedback Lab	Core	Input System/TMP should be optional or guarded; all runtime presenters must be layout-agnostic.

7. Runtime, Editor, Data, and Serialization Rules

Runtime package rules

- Runtime scripts must compile in a player build without UnityEditor.
- Runtime ScriptableObjects should contain data and pure runtime logic only.
- Do not store editor window state in runtime assets unless it is meaningful to gameplay/runtime use.
- Serialized fields should have tooltips and sane defaults.
- Use stable string IDs or GUID-like keys for anything referenced across assets, adapters, or package boundaries.

Editor package rules

- Editor windows should use SerializedObject and SerializedProperty when editing assets with existing references.
- Use Undo.RecordObject / Undo.RegisterCreatedObjectUndo for scene-object changes.
- Use AssetDatabase and EditorUtility.SetDirty safely for asset creation and mutation.
- Prompt before opening/changing scenes or applying destructive project-wide changes.
- Prefer explicit Scan, Preview, Apply Selected, Apply All flows for batch operations.

Migration and compatibility

- Avoid destructive runtime class renames for components already used in scenes or prefabs.
- When replacing a project-specific tool, prefer a new generic base plus legacy wrapper/alias rather than two parallel implementations.
- Migration installers should be temporary unless they provide durable package value.
- Class wrappers and menu aliases should preserve old workflows while directing users into the new generic implementation.

8. Performance and Safety Standards

Performance

- Do not scan all scenes/assets on every OnGUI repaint.
- Cache scan results and rebuild only when settings change or the user requests refresh.
- Use EditorApplication.update queues for long-running preview/icon/export work.
- Use cancelable progress bars for expensive scans.
- Avoid Thread.Sleep in editor tools.
- Avoid SceneView.RepaintAll unless broad repaint is truly necessary. Prefer the active window or active SceneView.
- Avoid LINQ-heavy filtering/sorting inside every OnGUI for large result sets.

Safety

- Dry-run or preview first for destructive and batch operations.
- Make dangerous toggles visually obvious and default them to safe behaviour.
- Do not overwrite existing user data unless explicitly enabled.
- Do not modify shared materials or prefab assets by default without a warning.
- When applying results, show a summary of changed, ignored, and failed items.

9. Current Archive Audit Snapshot

This section records durable facts from the latest uploaded PungentFunkUtilities.zip. It is an architectural snapshot rather than a progress plan. The script index below was regenerated against this archive.

Finding	Current audit result	Architectural meaning
C# scripts	94 scripts, 30,404 total C# lines.	The package is broad enough to require strict lab boundaries, namespace policy, and script ownership documentation.
Editor/runtime split	56 files under Editor; 38 runtime/shared files outside Editor.	The package has a useful structural split, but asmdefs are still required for clean export boundaries.
Path delta from previous indexed bible	0 new C# script paths and 0 removed C# script paths detected in this uploaded archive.	The current pass appears to change code/menu metadata rather than adding or removing C# file paths.
Namespace declarations	0 files with namespace declarations.	The package remains in the global namespace. Exported assets should add namespaces to reduce collisions.
Assembly definition files	0 asmdef files found.	Standalone assets should use runtime/editor asmdefs and optional bridge asmdefs.
MenuItem declarations	77 MenuItem attributes found.	Public access is broad and growing; menu cleanup is still required before release.
CreateAssetMenu declarations	12 CreateAssetMenu attributes found.	CreateAssetMenu roots are inconsistent and should be standardised before distribution.
InitializeOnLoad usage	Editor/Asset Placement Lab/PungentAssetPlacementUtilitiesRegistrar.cs (1); Editor/PungentTooltipNotesBrowserWindow.cs (1); Editor/PungentUtilityTrayWindow.cs (1)	Startup hooks should stay light, deterministic, and registry-consistent.
Runtime UnityEditor references	Modular Paths/ModularPathSpawner.cs	ModularPathSpawner still contains editor-only rebuild calls behind UNITY_EDITOR; split editor rebuild logic for asmdef cleanliness.

Finding	Current audit result	Architectural meaning
Optional TMP dependency	FontPreviewWindow has compile-time TMPPro usage; PaletteApplyUtility references TMP via reflection/string-style optional handling.	TMP must be declared, optionalized, or isolated in a bridge if the relevant package ships standalone.

Menu and public path audit

- The current archive mostly uses Tools/Utilities/... paths, with legacy aliases still present: Tools/RenameTool, Tools/Font Preview, Tools/Terrain/Terrain Usage Scanner, Tools/Debug/Debug Control Center, and Tools/Pungent Colour Suite/Open Palette Designer.

- Some context menu entries appear twice because validation overloads share the same display path; this is acceptable in Unity but should be checked for avoidable user-facing duplication.

- CreateAssetMenu roots currently include PungentFunk Utilities/Asset Placement, PungentFunk/Audio, Pungent Funk/Generation, and PungentFunk Utilities/Colour Palette. Standardise these under one public root before release.

CreateAssetMenu path audit

File	Menu name	File name
Asset Placement Lab/PungentPlacementAssetSetSO.cs	PungentFunk Utilities/Asset Placement/Asset Set	Placement Asset Set
Asset Placement Lab/PungentPlacementRuleSetSO.cs	PungentFunk Utilities/Asset Placement/Rule Set	Placement Rule Set
Audio Coverage/AudioClipSetSO.cs	PungentFunk/Audio/Clip Set	AudioClipSet
Audio Coverage/AudioCoverageProfileSO.cs	PungentFunk/Audio/Editor/Audio Coverage Profile	AudioCoverageProfile
Audio Coverage/AudioInteractionMatrixSO.cs	PungentFunk/Audio/Interaction Matrix	AudioInteractionMatrix
Audio Coverage/AudioInteractionProfileSO.cs	PungentFunk/Audio/Interaction Profile	AudioInteractionProfile
Audio Coverage/AudioMaterialLibrarySO.cs	PungentFunk/Audio/Material Library	AudioMaterialLibrary
Audio Coverage/AudioSurfaceMaterialSO.cs	PungentFunk/Audio/Surface Material	AudioSurfaceMaterial
Audio Coverage/TerrainAudioMaterialProfileSO.cs	PungentFunk/Audio/Terrain Material Profile	TerrainAudioMaterial Profile
Name Generator/MadLibNameList.cs	Pungent Funk/Generation/Mad-Lib Name List	New Mad-Lib Name List
Name Generator/NameList.cs	Pungent Funk/Generation/Name List	New Name List
Palette Designer/PungentColourPaletteSO.cs	PungentFunk Utilities/Colour Palette	Pungent Colour Palette

Scan/repaint/performance hotspot audit

Pattern	Files found	Meaning
SceneView.RepaintAll	Editor/Asset Placement Lab/AssetPlacementLabWindow.cs (3); Editor/PungentPathAuthoringToolkitWindow.cs (1); Editor/PungentTooltipNotesBrowserWindow.cs (1); Editor/PungentUtilityTrayWindow.cs (2); Editor/Scene Gizmos/PungentGizmoBrowserWindow.cs (1)	Keep only where broad scene refresh is necessary; otherwise repaint active window/SceneView.
Resources.FindObjectsOfType All	Editor/Debug Control Window/DebugControlWindow.cs (1); Editor/ReferenceAssignmentScannerWindow.cs (2); Editor/Scene Gizmos/PungentGizmoBrowserWindow.cs (1); Editor/Utilities Core/Window Themes/UtilityWindowTheme.cs (1)	Use explicit scan/cache patterns and avoid calling during every repaint.
AssetDatabase.FindAssets	Editor/Audio Coverage/AudioCoverageContextWindow.cs (3); Editor/Audio Coverage/AudioCoverageWindow.cs (3); Editor/Audio Coverage/AudioInteractionMatrixEditor.cs (1); Editor/FontPreviewWindow.cs (2); Editor/InputPromptIconLibraryAutoFill.cs (4); Editor/Name Generator/NameGeneratorWindow.cs (2); Editor/PungentCoverageMatrixWindow.cs (1); Editor/PungentTokenValidatorWindow.cs (1); Editor/ReferenceAssignmentScannerWindow.cs (1); Editor/TerrainUsageScannerWindow.cs (2)	Acceptable when user-triggered/cached; avoid automatic repeated project scans.
Thread.Sleep	Editor/PrefabIconGeneratorWindow.cs (1)	Replace editor blocking with update queues or async-style staged jobs.

Largest maintainability targets

Script	Lines	Architectural note
Editor/Debug Control Window/DebugControlWindow.cs	2,869	Candidate for panel/service/state split if future work increases complexity.
Editor/Palette Designer/PaletteDesignerWindow.cs	2,168	Candidate for panel/service/state split if future work increases complexity.
Editor/Audio Coverage/AudioCoverageWindow.cs	1,518	Candidate for panel/service/state split if future work increases complexity.
Modular Paths/ModularPathSpawner.cs	978	Candidate for panel/service/state split if future work increases complexity.
Editor/TerrainUsageScannerWindow.cs	953	Candidate for panel/service/state split if future work increases complexity.
Editor/ReferenceAssignmentScannerWindow.cs	880	Candidate for panel/service/state split if future work increases complexity.
Editor/Utilities Core/Window Themes/UtilityWindowTheme.cs	846	Candidate for panel/service/state split if future work increases complexity.
Editor/Utilities Core/Window Themes/UtilityWindowThemeCustomizer.cs	813	Candidate for panel/service/state split if future work increases complexity.
Editor/Audio Coverage/AudioCoverageContextWindow.cs	760	Candidate for panel/service/state split if future work increases complexity.
Editor/Name Generator/NameGeneratorWindow.cs	729	Candidate for panel/service/state split if future work increases complexity.
Editor/FontPreviewWindow.cs	659	Candidate for panel/service/state split if future work increases complexity.
Editor/Modular Paths/ModularPathWindow.cs	653	Candidate for panel/service/state split if future work increases complexity.

10. Lab Taxonomy

Lab	Product role	Primary ownership
Utility Core	Shared shell and UX infrastructure.	Registry, control panel, lab menus, tray, theme, preferences, editor performance helpers.
Debug Lab	Runtime/editor debug control and signal observation.	Debug channels, router, signal relay, reflected debug fields/actions, scheduled debug calls.
Asset Placement Lab	Reusable scene asset placement and procedural dressing.	Asset sets, rules, scatter/grid/socket/group workflows, placement results, scene application.
Scene Workflow Lab	Scene authoring speed tools.	Scene navigation, gizmo browsing/sources, surface alignment, path authoring helpers.
Colour Lab	Palette creation, analysis, accessibility, and application.	Palette assets, swatches, harmony, contrast, deficiency preview, palette generation and material/UI application.
Texture Lab	Procedural texture/mask generation and texture array baking.	Texture settings, procedural generator, texture-array source selection, bake validation.
Audio Lab	Surface/contact audio data and coverage validation.	Audio materials, clip sets, interaction profiles/matrices, terrain mappings, contact router/classifier, coverage scanners.
Asset Preview and Export Lab	Preview/render/export support for reusable assets.	Prefab icons, prefab asset extraction, font previews.
Project Audit and Authoring Lab	Generic authoring diagnostics and batch tools.	Reference assignment, terrain usage, tuning copy, rename, coverage matrix, token validation, tooltip notes.
Content Generation Lab	Reusable text/name generation.	Name lists, MadLib lists, name-generation window.

Lab	Product role	Primary ownership
UI and Feedback Lab	Reusable input prompt and UI feedback seed tools.	Current seed is icon library auto-fill; runtime presenters should remain optional and layout-agnostic.

11. Script Index

This index covers every C# script present in the latest audited archive. Paths are relative to the PungentFunkUtilities package root. Line counts were regenerated from the current zip. Purpose and feature summaries are architecture-level descriptions intended to remain useful as implementation details evolve.

Utility Core

Editor/PungentUtilityTrayWindow.cs (305 lines)

Purpose: Compact tray for parked/recent utility windows.

Feature set / links: Reopen or park focused utilities; interacts with registry and scene repainting. Debug Lab

Editor/Utilities Core/PungentEditorPerformanceUtility.cs (77 lines)

Purpose: Shared editor-performance helpers; currently provides incremental queue support for staged editor work.

Feature set / links: Core utility for long-running or chunked editor tasks; should be expanded carefully rather than duplicated per window.

Editor/Utilities Core/PungentUtilityAccessMenus.cs (136 lines)

Purpose: Context and global menu access points into the control panel and frequently used tools.

Feature set / links: Routes Asset and GameObject context actions to registered tools; should remain thin and registry-driven.

Editor/Utilities Core/PungentUtilityControlPanelWindow.cs (336 lines)

Purpose: Main utility launcher and lab/category browser.

Feature set / links: Search, recent tools, lab/category filtering, descriptor-driven opening, shared themed layout.

Editor/Utilities Core/PungentUtilityDescriptor.cs (162 lines)

Purpose: Serializable metadata contract for discoverable utilities.

Feature set / links: ID, display name, category, lab, module, description, menu path, tags, window type, capability flags, status, related IDs, icon, open action.

Editor/Utilities Core/PungentUtilityLabMenus.cs (39 lines)

Purpose: Direct menu routing for lab hub views.

Feature set / links: Tools/Utilities/Labs entries route into filtered control-panel views.

Editor/Utilities Core/PungentUtilityLabs.cs (22 lines)

Purpose: Central lab name constants.

Feature set / links: Prevents string drift for lab labels used by registry and launcher.

Editor/Utilities Core/PungentUtilityRegistry.cs (265 lines)

Purpose: Central registry for utility descriptors.

Feature set / links: Default registration, duplicate-ID replacement, lab/category/module queries, sort order, descriptor opening.

Editor/Utilities Core/Window Themes/UtilityWindowPrefs.cs (35 lines)

Purpose: EditorPrefs wrapper for shared window state.

Feature set / links: Booleans, ints, floats, and strings under consistent preference keys.

Editor/Utilities Core/Window Themes/UtilityWindowTheme.cs (846 lines)

Purpose: Shared ImGui visual system.

Feature set / links: Headers, panels, pills, toolbar toggles, tinted buttons, foldout styling, resize handles, role colours, theme presets, deficiency preview roles.

Editor/Utilities Core/Window Themes/UtilityWindowThemeCustomizer.cs (813 lines)

Purpose: Editor window for modifying the shared theme.

Feature set / links: Preset selection, custom palette records, harmony/randomization controls, role colours, persistence.

Debug Lab

Editor/Debug Control Window/DebugChannels.cs (65 lines)

Purpose: Shared debug channel and signal constants.

Feature set / links: Defines reusable channel/signal naming conventions for debug routing.

Editor/Debug Control Window/DebugControlWindow.cs (2,869 lines)

Purpose: Large debug control center.

Feature set / links: Discovers debug flags and component state; toggles static fields; schedules reflected actions; provides router view and component inspector tooling.

Editor/Debug Control Window/DebugRouter.cs (626 lines)

Purpose: Runtime/editor debug message router.

Feature set / links: Channels, source/state/signal snapshots, levels, subscriptions, source ownership, and diagnostics history.

Editor/Debug Control Window/DebugSignalRelay.cs (86 lines)

Purpose: Scene component for relaying debug signals to UnityEvents.

Feature set / links: String-event rules let scene objects react to debug signals without project-specific dependencies. Asset Placement Lab Runtime

Asset Placement Lab

Asset Placement Lab/PungentPlacedAssetMarker.cs (15 lines)

Purpose: Marker component attached to generated placement instances.

Feature set / links: Stores group ID, module ID, seed, source prefab, estimated bounds, and generation timestamp.

Asset Placement Lab/PungentPlacementAssetSetSO.cs (153 lines)

Purpose: Weighted prefab catalog for placement workflows.

Feature set / links: Prefab entries, tags, probability/weight, random scale, random rotation, footprint metadata, and random selection.

Asset Placement Lab/PungentPlacementBoundsUtility.cs (114 lines)

Purpose: Bounds helper for placement workflows.

Feature set / links: Hierarchy bounds, prefab local/world bounds estimation, absolute extents, and selection encapsulation.

Asset Placement Lab/PungentPlacementGridUtility.cs (68 lines)

Purpose: Grid and footprint helper methods.

Feature set / links: World-to-cell, cell-to-world, snapping, rotated footprint cells, overlap checks, and occupancy marking.

Asset Placement Lab/PungentPlacementGroup.cs (25 lines)

Purpose: Root metadata component for generated groups.

Feature set / links: Group ID, display name, module, seed, source asset set/rules, and child counting.

Asset Placement Lab/PungentPlacementRuleSetSO.cs (81 lines)

Purpose: Reusable placement rule profile.

Feature set / links: Surface masks, spacing, height/slope filtering, heatmap sampling, overlap settings, grouping, naming, and marker behavior.

Asset Placement Lab/PungentPlacementScatterUtility.cs (337 lines)

Purpose: Candidate generation engine for scatter placement.

Feature set / links: Random, min-spacing, grid, jittered grid, hex, spiral, ring, and line candidate generation with raycast/rule rejection states.

Asset Placement Lab/PungentPlacementSocket.cs (42 lines)

Purpose: Socket metadata for modular prefab connection points.

Feature set / links: Socket ID, category, accepted categories, size, direction, and compatibility checks.

Asset Placement Lab/PungentPlacementTypes.cs (190 lines)

Purpose: Placement enums and data transfer models.

Feature set / links: Scatter patterns, area modes, surface alignment, rejection reasons, surface categories, footprint, tag list, candidate, result, and context models. Asset Placement Lab - Editor

Editor/Asset Placement Lab/AssetPlacementLabWindow.cs (618 lines)

Purpose: Main Asset Placement Lab window.

Feature set / links: Area Scatter, Grid/Footprint, Socket Validator, and Group Manager modules; scene preview; rule summaries; asset-set builder; results application.

Editor/Asset Placement Lab/PungentAssetPlacementAccessMenus.cs (34 lines)

Purpose: Asset/GameObject context menu helpers for placement.

Feature set / links: Create asset set from selected prefabs; add placement socket to selected objects.

Editor/Asset Placement Lab/PungentAssetPlacementUtilitiesRegistrar.cs (22 lines)

Purpose: Current standalone placement-lab registry hook.

Feature set / links: InitializeOnLoad registration for Asset Placement Lab; should use full lab/module metadata or be folded into consistent per-lab registration.

Editor/Asset Placement Lab/PungentPlacementApplyUtility.cs (130 lines)

Purpose: Applies placement results to the active scene.

Feature set / links: Prefab instantiation, group root creation, Undo support, marker attachment, naming, and scene dirtying.

Editor/Asset Placement Lab/PungentPlacementAssetSetBuilder.cs (83 lines)

Purpose: Creates placement asset sets from selected prefabs.

Feature set / links: Selection import, asset creation, entry setup, and AssetDatabase persistence.

Editor/Asset Placement Lab/PungentPlacementSocketValidator.cs (79 lines)

Purpose: Socket diagnostics and setup helper.

Feature set / links: Validates missing/duplicate socket IDs, empty categories, invalid size/direction, and can add sockets to selected objects.
Scene Workflow Lab

Scene Workflow Lab

Editor/Modular Paths/ModularPathSpawnerEditor.cs (305 lines)

Purpose: Custom editor and scene handles for ModularPathSpawner.

Feature set / links: Point handles, add/remove/select controls, delayed editor rebuilds, and inspector support.

Editor/Modular Paths/ModularPathWindow.cs (653 lines)

Purpose: Dedicated Modular Path Builder window.

Feature set / links: Create/manage path spawners, edit points, configure generated prefabs/surface rules, resize panels, and route scene-handle workflows. Colour Lab

Editor/PungentPathAuthoringToolkitWindow.cs (385 lines)

Purpose: Path authoring utility and toolkit.

Feature set / links: Path edit modes, handle/waypoint helpers, sampling/validation concepts, and scene overlay guidance for path-based tools.

Editor/PungentSceneHelpOverlay.cs (21 lines)

Purpose: Lightweight scene help overlay hook.

Feature set / links: SceneView overlay-style help entry point for authoring guidance.

Editor/PungentSceneNavigationWindow.cs (558 lines)

Purpose: Scene navigation and waypoint window.

Feature set / links: Saved waypoints, SceneView fast travel, selected object tracking, conditional trackers, context actions, and scene GUI overlay.

Editor/Scene Gizmos/PungentGizmoBrowserWindow.cs (174 lines)

Purpose: Scene gizmo browser.

Feature set / links: Cached source scanning, include inactive option, search, focus/select/toggle helpers, and gizmo-source management.

Editor/Scene Gizmos/PungentSceneGizmoSourceEditor.cs (207 lines)

Purpose: Custom inspector for scene gizmo sources.

Feature set / links: Rule editing, template insertion, preview controls, and clearer source authoring.

Editor/TerrainAlignTool.cs (489 lines)

Purpose: Surface Align Tool plus legacy Terrain Align compatibility.

Feature set / links: Align selected objects to collider/surface raycasts with optional normal alignment, offsets, previews, and legacy window alias.

Modular Paths/ModularPathSpawner.cs (978 lines)

Purpose: Runtime modular path generator component.

Feature set / links: Control points, polyline/Catmull-Rom sampling, socket placement, prefab length estimation, surface conformance, rebuild, generated root, and legacy fence terminology wrappers.

PungentSceneGizmoSource.cs (493 lines)

Purpose: Runtime scene gizmo source component.

Feature set / links: Configurable gizmo rules, conditions, labels, shapes, runtime/editor draw modes, object value resolution, and templates.

Colour Lab

Editor/Palette Designer/PaletteApplyUtility.cs (273 lines)

Purpose: Palette application helper.

Feature set / links: Finds materials/renderers/UI/text targets, applies palette colours to selected assets/objects, and uses reflection for optional TMP-style text support.

Editor/Palette Designer/PaletteDesignerWindow.cs (2,168 lines)

Purpose: Main palette authoring window.

Feature set / links: Palette board, swatch editing, generation panel, selected swatch details, analysis, application tools, palette save/load, advanced options, and resizable sections.

Editor/Palette Designer/PungentColourPaletteSOEditor.cs (25 lines)

Purpose: Custom editor for palette assets.

Feature set / links: Inspector integration for palette data.

Editor/Palette Designer/PungentPaletteStorageUtility.cs (153 lines)

Purpose: Palette persistence helper.

Feature set / links: Default asset paths, save/load/create workflows, and palette asset management. Texture Lab

Palette Designer/ColourContrastUtility.cs (74 lines)

Purpose: Contrast helpers.

Feature set / links: Relative luminance, contrast ratios, readability ratings, and foreground/background scoring.

Palette Designer/ColourConversionUtility.cs (196 lines)

Purpose: Colour mode conversion utilities.

Feature set / links: RGB/HSV/CMYK/LAB/grayscale-style value conversions and mode value containers.

Palette Designer/ColourDeficiencyPreviewUtility.cs (67 lines)

Purpose: Colour deficiency simulation.

Feature set / links: Preview modes and matrix-style transforms for colour blindness review.

Palette Designer/ColourHarmonyUtility.cs (97 lines)

Purpose: Harmony calculations.

Feature set / links: Complementary, split, analogous, triadic, tetradic/square-like hue generation and colour harmony helpers.

Palette Designer/PaletteAnalysisUtility.cs (167 lines)

Purpose: Palette analysis and reporting.

Feature set / links: Contrast pairs, role coverage, readability diagnostics, duplicate/similarity concerns, and analysis report models.

Palette Designer/PaletteGenerationSettings.cs (77 lines)

Purpose: Palette generation settings model.

Feature set / links: Harmony mode, seed, count, role/tone/contrast controls, and generator configuration.

Palette Designer/PaletteGeneratorUtility.cs (302 lines)

Purpose: Palette generation engine.

Feature set / links: Harmony-aware colour generation, swatch naming, locked swatch preservation, variation generation, and role assignment support.

Palette Designer/PaletteSwatch.cs (58 lines)

Purpose: Serializable palette swatch model.

Feature set / links: Name, colour, role, lock state, priority, clone support, and palette role enum.

Palette Designer/PungentColourPaletteSO.cs (79 lines)

Purpose: Reusable colour palette asset.

Feature set / links: Palette name, description, tags, swatches, add/remove/find operations, and clone-style use by editor tools.

Texture Lab

Editor/Procedural Texture Lab/ProceduralTextureLabWindow.cs (425 lines)

Purpose: Procedural Texture Lab editor window.

Feature set / links: Settings UI, live preview, generation/export, density/stamp texture fields, seed randomization, and preview resizing.

Editor/Texture Array Baker/TextureArrayBakerUtility.cs (247 lines)

Purpose: Texture2DArray bake utility.

Feature set / links: Source validation, fallback slices, material property extraction, texture resizing/conversion, and result reporting.

Editor/Texture Array Baker/TextureArrayBakerWindow.cs (451 lines)

Purpose: Texture array baker window.

Feature set / links: Texture/material source modes, drag-and-drop source list, custom shader property search, preview, bake output, and fallback controls. Audio Lab - Runtime

Generation/ProceduralTextureGenerator.cs (599 lines)

Purpose: Procedural texture generation engine.

Feature set / links: Pattern point generation, stamp drawing, blend modes, gradients/foreground-background output, density map handling, and texture creation.

Generation/ProceduralTextureSettings.cs (171 lines)

Purpose: Runtime-safe procedural texture settings.

Feature set / links: Output, seed, pattern, stamp, colour, grid, hex, Poisson, density-map, polar, texture-stamp settings and clamping.

Audio Lab

Audio Coverage/AudioClipSetSO.cs (148 lines)

Purpose: Reusable clip-set asset.

Feature set / links: Weighted or sequential clip selection, volume/pitch ranges, cooldowns, and immediate-repeat avoidance.

Audio Coverage/AudioCoverageProfileSO.cs (509 lines)

Purpose: Audio setup coverage profile asset.

Feature set / links: Cue binding rules, component expectations, required fields, runtime hook type names, ignored cues, scene/prefab scan options, and generic defaults.

Audio Coverage/AudioInteractionMatrixSO.cs (87 lines)

Purpose: Surface interaction lookup matrix.

Feature set / links: Exact material-pair profiles, category fallback rules, and default fallback profile resolution.

Audio Coverage/AudioInteractionProfileSO.cs (66 lines)

Purpose: Audio profile for material interactions.

Feature set / links: Contact-event clip rules and blend settings for material pairs.

Audio Coverage/AudioMaterialLibrarySO.cs (35 lines)

Purpose: Surface-audio material library.

Feature set / links: Find/contains/get-all access for AudioSurfaceMaterialSO entries.

Audio Coverage/AudioMaterialResolver.cs (115 lines)

Purpose: Runtime resolver from colliders/raycast hits/terrain to audio materials.

Feature set / links: AudioMaterialTag lookup, terrain dominant/weighted material lookup, and surface resolution helpers.

Audio Coverage/AudioMaterialTag.cs (65 lines)

Purpose: Collider/object audio material tag.

Feature set / links: Primary/secondary material assignment, role metadata, child collider application, and try-get helpers.

Audio Coverage/AudioSurfaceMaterialSO.cs (71 lines)

Purpose: Surface audio material definition.

Feature set / links: ID, display name, category, hardness/softness helpers, and debug labels.

Audio Coverage/ContactAudioEvent.cs (19 lines)

Purpose: Contact audio event data structure.

Feature set / links: Stores material pair, point, normal, speed, intensity, duration, source object/collider, sustained state, and contact ID for routing/classification.

Audio Coverage/ContactAudioEventType.cs (13 lines)

Purpose: Contact event type enum.

Feature set / links: None, brush/slide/drag/scrape, light/medium/heavy impact, bounce, release, and related event classes.

Audio Coverage/ContactAudioRouter.cs (206 lines)

Purpose: Runtime contact audio playback router.

Feature set / links: Profiles/matrices, transient playback, sustained loop handling, AudioSource pooling, rule resolution, intensity updates, and cleanup.

Audio Coverage/ContactEventClassifier.cs (107 lines)

Purpose: Runtime contact classifier.

Feature set / links: Converts raw collision/contact information into typed audio events using impact, tangential speed, bounce, and duration thresholds.

Audio Coverage/ContactLoopInstance.cs (63 lines)

Purpose: Runtime sustained-loop state object.

Feature set / links: Initialize/update/fade/expire/stop lifecycle for looping contact audio sources.

Audio Coverage/TerrainAudioMaterialProfileSO.cs (135 lines)

Purpose: TerrainLayer-to-audio-material profile.

Feature set / links: Layer bindings, dominant material resolution, weighted material blending, and fallback material support. Audio Lab - Editor

Editor/Audio Coverage/AudioCoverageContextWindow.cs (760 lines)

Purpose: Audio setup coverage scanner.

Feature set / links: Validates scenes/prefabs/assets against AudioCoverageProfileSO expectations; reports severity, context, missing fields, runtime hook coverage, terrain profiles, and interaction matrices.

Editor/Audio Coverage/AudioCoverageProfileSOEditor.cs (49 lines)

Purpose: Custom editor for AudioCoverageProfileSO.

Feature set / links: Reset-to-defaults and profile inspector enhancements.

Editor/Audio Coverage/AudioCoverageWindow.cs (1,518 lines)

Purpose: Audio catalog coverage scanner.

Feature set / links: Audits cue/catalog/binding coverage, script references, duplicate/missing/incomplete cue entries, setup helpers, and catalog-profile relationships.

Editor/Audio Coverage/AudioInteractionMatrixEditor.cs (391 lines)

Purpose: Custom editor for interaction matrices.

Feature set / links: Missing pair diagnostics, duplicate checks, fallback/library validation, and matrix authoring help.

Editor/Audio Coverage/AudioMaterialTagEditor.cs (150 lines)

Purpose: Custom editor for AudioMaterialTag.

Feature set / links: Tag inspector with child-collider application and material role guidance.

Editor/Audio Coverage/TerrainAudioProfileEditor.cs (167 lines)

Purpose: Custom editor for terrain audio profiles.

Feature set / links: Binding diagnostics, duplicate TerrainLayer checks, and profile summary. Asset Preview and Export Lab

Asset Preview and Export Lab

Editor/FontPreviewWindow.cs (659 lines)

Purpose: Font and TMP font preview window.

Feature set / links: Search/filter fonts, preview cached text renders, queue preview generation, resize preview settings, and compare Unity/TMP font assets.

Editor/Prefab Exporter/PrefabAssetExporter.cs (291 lines)

Purpose: Prefab asset extraction service.

Feature set / links: Copies/reuses meshes/materials/textures used by prefab hierarchies into export folders, supports skinned meshes, and reports saved/reused assets.

Editor/Prefab Exporter/PrefabAssetExporterWindow.cs (107 lines)

Purpose: Prefab Asset Exporter window.

Feature set / links: Selection source, output folder, include disabled/skinned options, material/texture copy controls, and export report display. Project Audit and Authoring Lab

Editor/PrefabIconGeneratorWindow.cs (372 lines)

Purpose: Prefab icon generator window and service.

Feature set / links: Non-blocking preview-to-PNG jobs, queue/cancel workflow, selection support, and asset icon export.

Project Audit and Authoring Lab

Editor/BulkRenameWindow.cs (651 lines)

Purpose: Bulk rename tool.

Feature set / links: Manual targets, drag-and-drop, selection import, pattern/replace/numbering modes, preview, apply, and legacy menu alias.

Editor/ComponentTuningCopyWindow.cs (535 lines)

Purpose: Generic component tuning copier.

Feature set / links: Match source/target components, protect target-specific references, dry-run/copy modes, serialized property copying, and summary stats.

Editor/PungentCoverageMatrixWindow.cs (313 lines)

Purpose: Generic coverage matrix tool.

Feature set / links: Manual or asset-derived dimensions, status/notes, gaps/duplicates, coverage templates, and scan/import support.

Editor/PungentTokenValidatorWindow.cs (210 lines)

Purpose: Token validation utility.

Feature set / links: Token definitions, pasted text or TextAsset scanning, required/optional token validation, and result reporting.

Editor/PungentTooltipNotesBrowserWindow.cs (542 lines)

Purpose: Custom tooltip/note browser.

Feature set / links: Notes by component/property/scope, priorities, visibility, create/edit/delete, selection context, and SceneView repaint hooks.

Editor/ReferenceAssignmentScannerWindow.cs (880 lines)

Purpose: Reference assignment scanner.

Feature set / links: Finds missing assignable references, scans scene/prefab/selection contexts, suggests candidates by type/name/path, preview/apply rows, and safety filtering.

Editor/TerrainUsageScannerWindow.cs (953 lines)

Purpose: Terrain usage scanner.

Feature set / links: Scans scenes and TerrainData assets, tracks used/unused terrain assets, configurable package exclusions, split result panels, and ping/select actions. Content Generation Lab

Content Generation Lab

Editor/Name Generator/NameGeneratorWindow.cs (729 lines)

Purpose: Name generation window.

Feature set / links: List and MadLib modes, token bindings, source list management, generated result list, uniqueness filtering, and copy/export style workflow.

Editor/Name Generator/NameListEditor.cs (116 lines)

Purpose: Custom editors for name-list assets.

Feature set / links: Inspector support for NameList and MadLibNameList assets.

Name Generator/MadLibNameList.cs (180 lines)

Purpose: Tokenised Mad-Lib name-list asset.

Feature set / links: Pattern fragments/tokens and list-backed generation for richer composed names.

Name Generator/NameList.cs (127 lines)

Purpose: Simple reusable name-list asset.

Feature set / links: Entries, optional weighting/metadata, random selection support, and CreateAssetMenu path.

UI and Feedback Lab

Editor/InputPromptIconLibraryAutoFill.cs (500 lines)

Purpose: Input prompt icon library populator.

Feature set / links: Reflection-free serialized key filling, default keyboard/mouse/gamepad entry map, sprite search, missing report, clear/fill options, and selected/default library commands.

12. Package Hardening Rules

Namespaces

Add namespaces before asset-store style distribution. Recommended root namespaces:

- PungentFunk.Utilities
- PungentFunk.Utilities.Runtime
- PungentFunk.Utilities.Editor
- PungentFunk.Utilities.Editor.Core
- PungentFunk.Utilities.Editor.Theme

- PungentFunk.Utilities.Placement
- PungentFunk.Utilities.Audio
- PungentFunk.Utilities.Colour
- PungentFunk.Utilities.Generation
- PungentFunk.Utilities.SceneTools
- PungentFunk.Utilities.UI

Assembly definitions

- Create separate runtime and editor asmdefs for each releasable lab or for Core + all labs if shipping as a single bundle first.
- Editor asmdefs reference runtime asmdefs. Runtime asmdefs never reference editor asmdefs.
- Optional bridges get their own asmdefs and dependencies, e.g. PungentFunk.Utilities.Placement.ColourBridge.
- Third-party dependencies such as TextMeshPro should be declared, optionalized, or isolated behind reflection/bridge code.

Status labels

Every utility descriptor should display a package status. Use these labels consistently: Stable, Experimental, In Progress, Deprecated, Project Adapter. These labels should drive both launcher badges and documentation expectations.

Documentation standard

- Each lab needs an overview, setup checklist, core concepts, recipes, troubleshooting, optional integration notes, and API/adapters page.
- Every editor window should have a Help/Notes foldout summarising what the tool changes and what it never changes.
- Every destructive or batch workflow should document its Undo support and its limits.

13. Utility Definition of Done

- Registered in PungentUtilityRegistry or consistent per-lab registrar with complete metadata.
- Primary menu path under Tools/Utilities/... or the final PungentFunk Utilities root, with optional context menus under Assets/GameObject/PungentFunk Utilities/... .
- Uses UtilityWindowTheme and UtilityWindowPrefs for visual style and persistent panel state.
- Responsive at small window sizes; no avoidable horizontal scrollbar; draggable splitters where panels compete for space.
- All fields have tooltips; enum choices have contextual descriptions; complex operations have inline help.
- Runtime code compiles without UnityEditor; editor code lives in Editor folders or editor asmdefs.
- No compile-time dependency on game-specific classes.
- Optional integrations degrade gracefully when missing.
- Scan-heavy tools cache results and refresh explicitly.
- Batch/destructive operations provide preview/dry-run and Undo or a clear warning where Undo is impossible.
- Tested in a clean project, with no optional labs installed, and with the full bundle installed.

14. Closing Doctrine

PungentFunk Utilities should remain modular in code, cohesive in user experience, and conservative in dependencies. The suite can feel interconnected without becoming tightly coupled. Core provides discovery and shared UX. Labs provide focused capability. Bridges provide optional collaboration. Project adapters preserve game-specific workflows without contaminating the generic assets.

When in doubt, choose the smaller generic utility, the thinner adapter, the safer apply flow, the clearer layout, and the more explicit dependency boundary.